CS 391
Web Application Development

Esma Meral

esma.meral@ozyegin.edu.tr

Week - 10

Outline

1. Routing

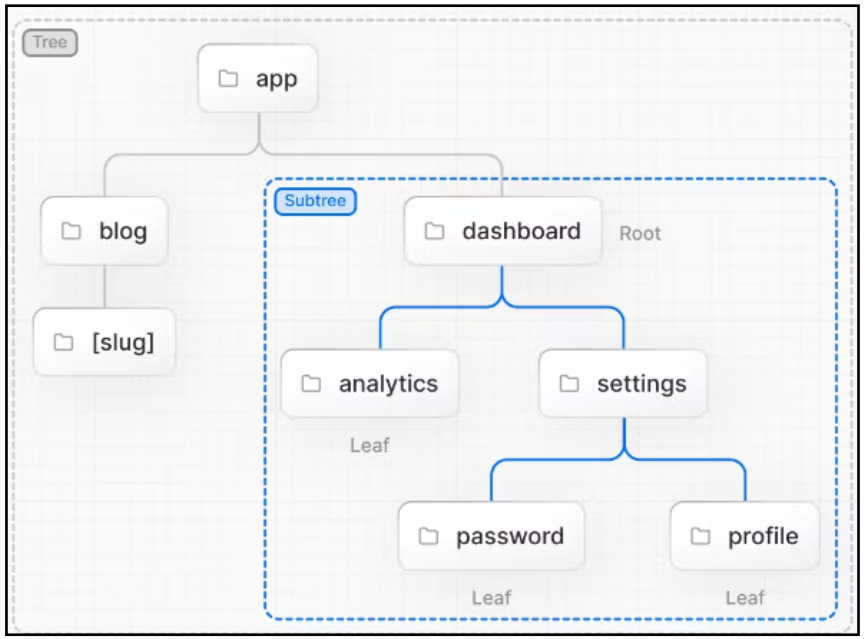
What is Routing?

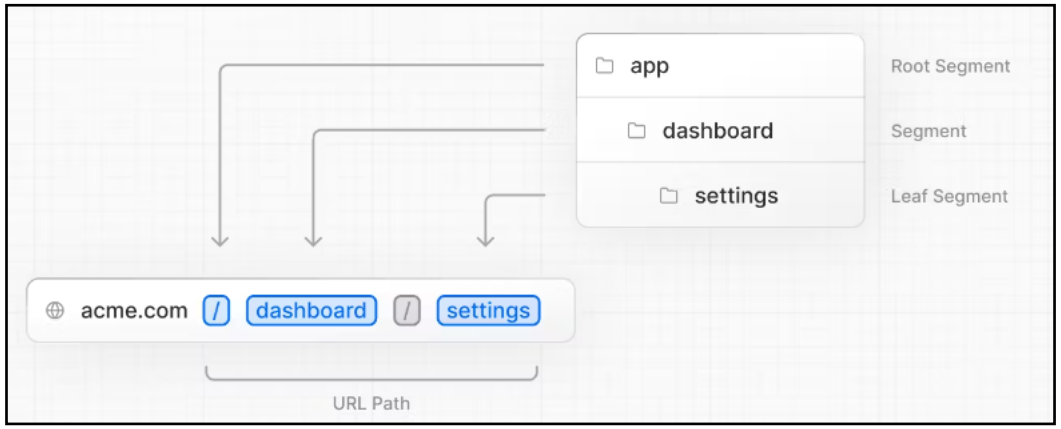
- In a short version, it's the concept of **mapping URL patterns to parts of your application**.
- Creating different URLs for different content
  - Enables the user to bookmark URLs
  - Single Page Applications(SPA)

```
// Express Style routing:
app.get("/", homePage);
app.get("/invoices", invoiceListPage);
app.get("/invoices/:id", invoicePage);
app.get("/invoices/:id/edit", invoiceEditPage);
```

Routing Fundamentals

- The skeleton of every application is routing.
- For example, a component tree with parent and child components, a folder structure, etc.





Routing Fundamentals

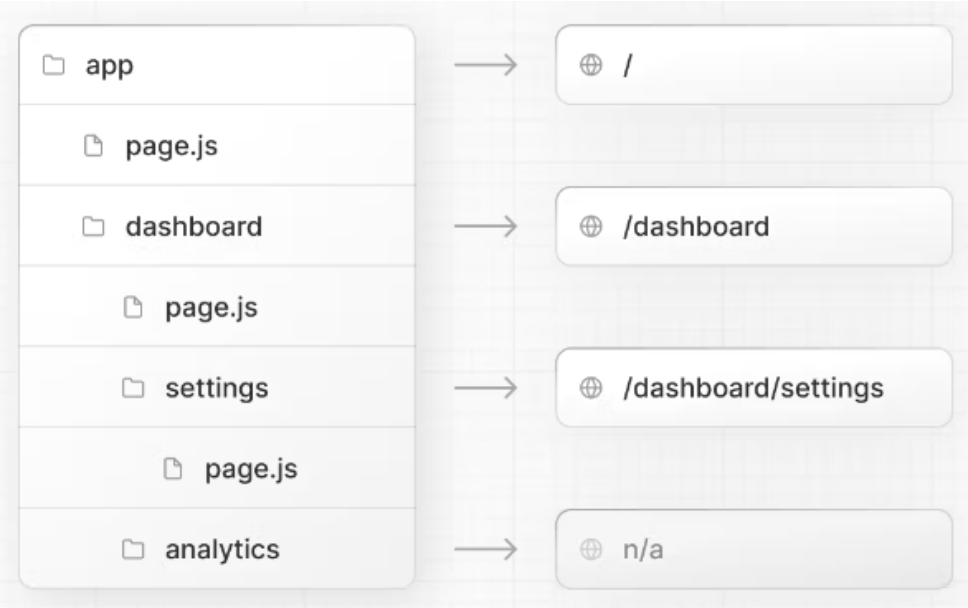
- Next.js uses a **file-system** based router where:
  - **Folders** are used to define routes.
  - **A route** is a single path of nested folders, following the file-system hierarchy from the root folder down to a final leaf folder that includes a page.js file.
  - **Files** are used to create UI that is shown for a route segment.

Reserved Filenames

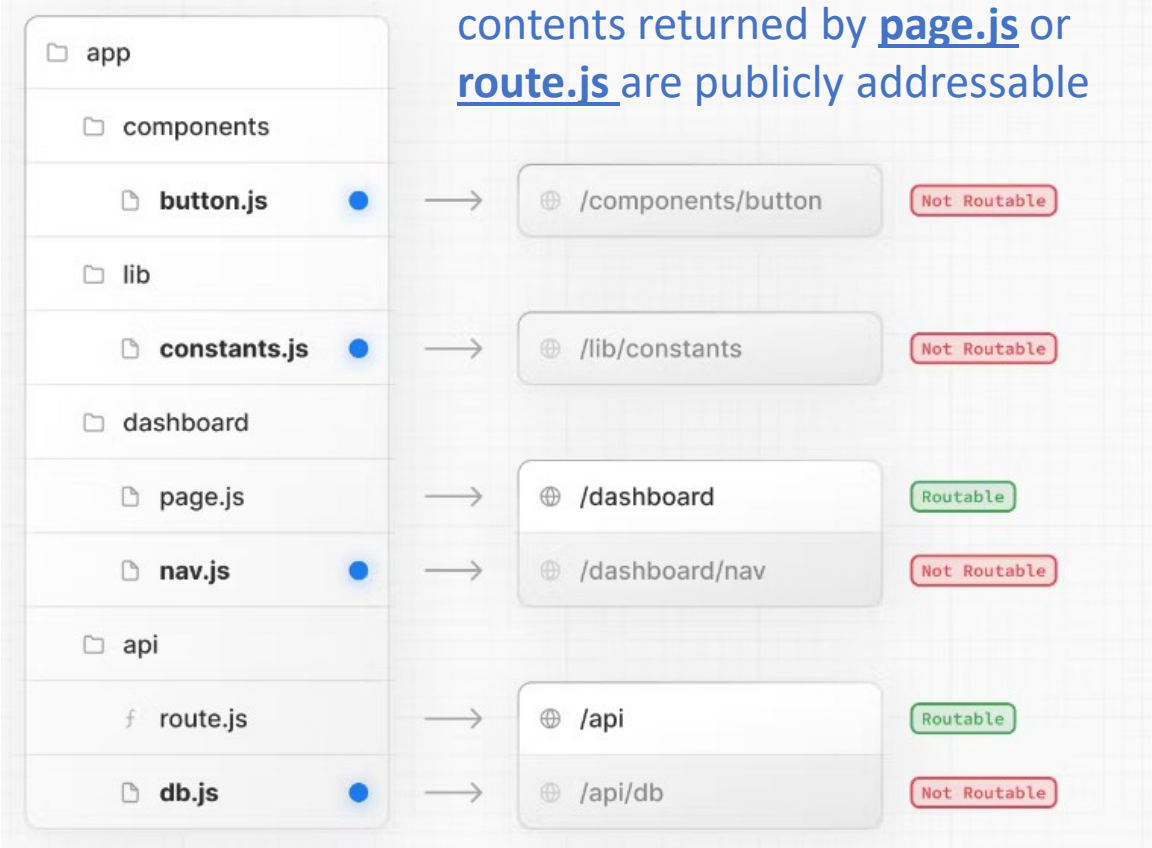
- There are some reserved filenames when working with NextJS.
- Important: These filenames are only reserved when creating them inside of the app/ folder (or any subfolder). Outside of the app/ folder, these filenames are not treated in any special way.
  - **page.js** => Create a new page (e.g., app/about/page.js creates a <your-domain>/about page)
  - **layout.js** => Create a new layout that wraps sibling and nested pages
  - **not-found.js** => Fallback page for "Not Found" errors
  - **error.js** => Fallback page for other errors
  - **loading.js** => Fallback page which is shown whilst sibling or nested pages (or layouts) are fetching data
  - **route.js** => Allows you to create an API route

Routing Fundamentals

- A special **page.js** file is used to make route segments publicly accessible.

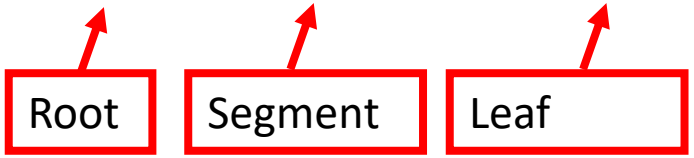


Folders define routes, only the contents returned by **page.js** or **route.js** are publicly addressable



Nested Routes

- To create a nested route, you can nest folders inside each other.
- For example, you can add a new /dashboard/settings route by nesting two new folders in the app directory.
- The /dashboard/settings route is composed of three segments:



Pages

- A page is UI that is unique to a route.
- You can define a page by **default** exporting a component from a page.js file.

app/page.js

```
export default function Page() {
  return <h1>Hello, Home page!</h1>
}
```

app/dashboard/page.js

```
export default function Page() {
  return <h1>Hello, Dashboard Page!</h1>
}
```

Layouts

- A layout is UI that is shared between multiple routes.
- You can define a layout by default exporting a React component from a **layout.js** file.
- The component should accept a **children prop** which can be a page or another layout.

Root Layout (Required)

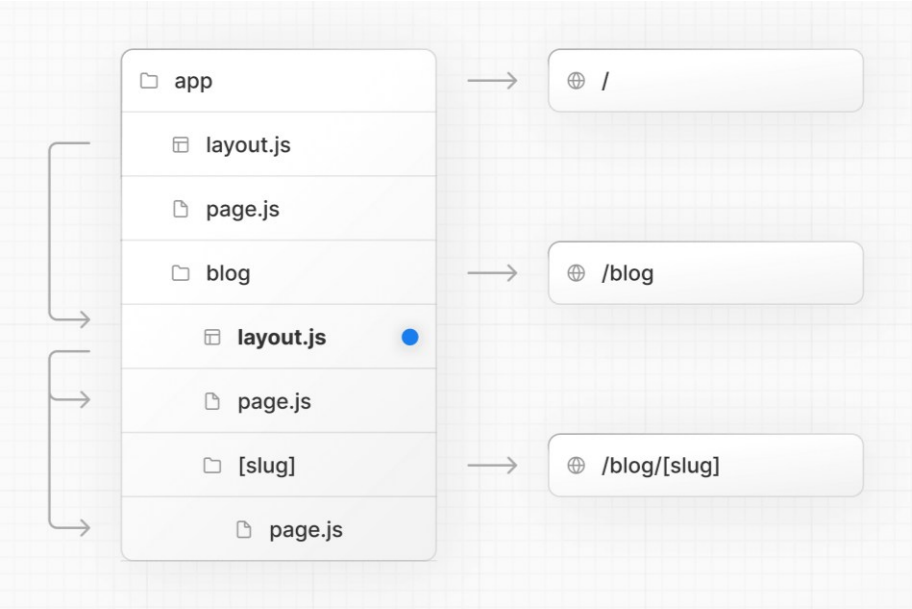
- The root layout is defined at the top level of the app directory and applies to all routes.
- This layout is required and must contain html and body tags, allowing you to modify the initial HTML returned from the server.

app/layout.js

```
export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>{children}</body>
    </html>
  );
}
```

Nesting Layouts

- By default, layouts in the folder hierarchy are nested, which means they **wrap** child layouts via their children prop.
- You can nest layouts by adding **layout.js** inside specific route segments (folders).
  - For example, to create a layout for the /blog route, add a new layout file inside the blog folder.



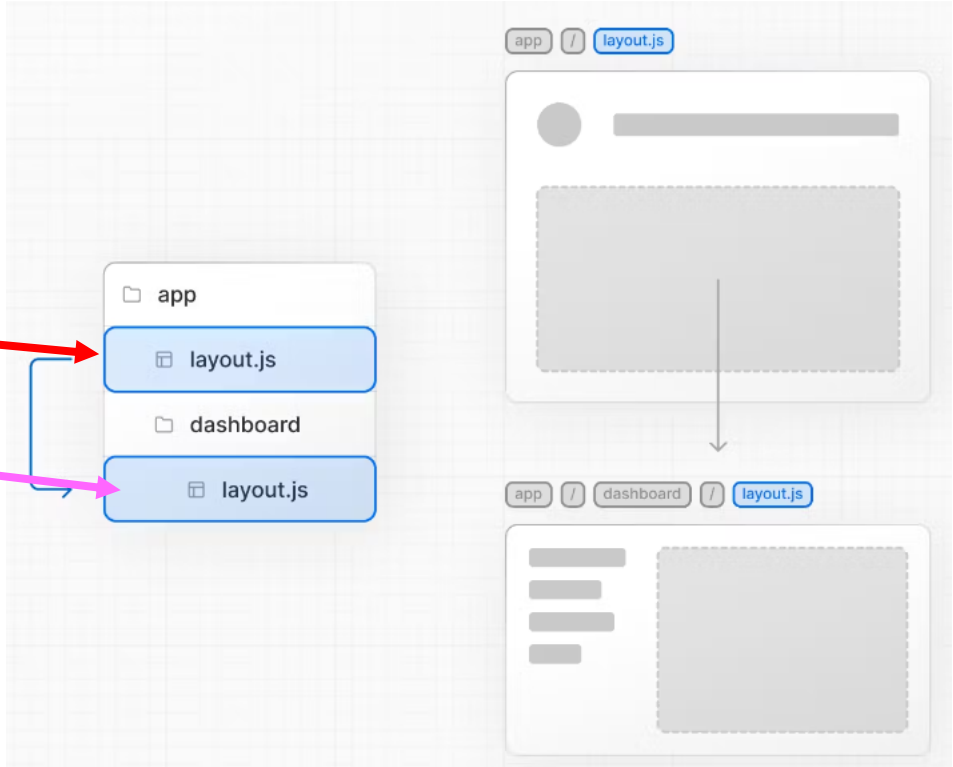
app/blog/layout.js

```
export default function BlogLayout({ children }) {
return <section>{children}</section>}
```

Nesting Layouts

- If you were to combine the two layouts above, the root layout would wrap the dashboard layout which would wrap route segments inside app/dashboard/*.

- The two layouts would be nested as such



Linking and Navigating

- There are four ways to navigate between routes in Next.js:
  - Using the <Link> Component
  - Using the useRouter hook (Client Components)
  - Using the redirect function (Server Components)
  - Using the native History API

<Link> Component

- <Link> is a built-in component that extends the HTML <a> tag to provide prefetching and client-side navigation between routes.
- Primary and recommended way to navigate between routes in Next.js.

```
import Link from 'next/link'

export default function Page() {
  return <Link href="/dashboard">Dashboard</Link>
}
```

useRouter() hook

- The useRouter hook allows you to programmatically change routes from Client Components.

```
'use client'

import { useRouter } from 'next/navigation'

export default function Page() {
  const router = useRouter()

  return (
    <button type="button" onClick={() => router.push('/dashboard')}>
      Dashboard
    </button>
  )
}
```

Dynamic Routes

- When you don't know the exact segment names ahead of time and want to create routes from dynamic data, you can use Dynamic Segments that are filled in at request time or prerendered at build time.
- A Dynamic Segment can be created by wrapping a folder's name in square brackets: [folderName]. For example, [id] or [isbn].
- Dynamic Segments are passed as the params prop to layout, page, route, and generateMetadata functions

Dynamic Routes

Route	Example URL	params
app/pages/catalog/[productid]/page.js	/pages/catalog/56	{ productid: 56 }
app/pages/catalog/[productid]/page.js	/pages/catalog/123	{ productid: 123 }
app/pages/catalog/[productid]/page.js	/pages/catalog/30981	{ productid: 30981 }

app/pages/catalog/[productid]/page.js

```
import { use } from 'react'
export default function ProductDetailPage({ params }) {
  const { productid } = use(params)
  return <div>Details:{productid} </div>
}
```

For versions:

```
"next": "16.2.1"
"react":"19.2.4"
```

Linking to Dynamic Routes

- For dynamic routes, it can be handy to use template literals to create the link's path.

```
<Link href={`/pages/${item.id}`}>{item.name}</Link>
```

- or

```
<Link href={"/pages/catalog/"+item.id}>{item.name}</Link>
```

Any Question?

Thank you for your attention

esma.meral@ozyegin.edu.tr